Министерство образования Республики Беларусь

Учреждение образования

«Брестский государственный технический университет»

Кафедра ИИТ

Лабораторная работа №4

По дисциплине «Модели решения задач в интеллектуальных системах»

Тема: «Классификация линейно разделимых образов однослойным персептроном для логических функций n переменных»

Выполнил:

Студент 3 курса

Группы ИИ-26

Петручик Д.А.

Проверила:

Андренко К.В.

Брест 2026

Цель работы: Реализовать нейронную сеть (однослойный персептрон) для решения задач синтеза логических функций от n входов. Сравнить эффективность обучения при различных начальных условиях и параметрах шага.

Задачи лабораторной работы:

1. Сгенерировать полную таблицу истинности для заданной логической функции (2n наборов). На выходе функции должны использоваться значения 0 и 1.
2. **Реализовать алгоритм обучения:**
   - Разделить выборку на обучающую (часть данных) и тестовую (оставшиеся наборы) для проверки обобщающей способности.
   - В качестве функции активации использовать сигмоиду
   - Провести **два эксперимента** по обучению:
     А. С фиксированным шагом обучения
     Б. С адаптивным шагом
   - Выход сети интерпретировать как вероятность принадлежности к классу «1».
   - Обучение проводить до минимизации суммарной ошибки на обучающей выборке.
3. **Провести анализ сходимости:**
   - Построить графики зависимости суммарной ошибки от номера эпохи для обучающей и тестовой выборок.
   - Оценить обобщающую способность: корректно ли сеть предсказывает значения на тех наборах из таблицы истинности, которые она не видела при обучении.
   - Зафиксировать количество эпох, необходимых для настройки весов при разных значениях
4. Вывести итоговые значения весовых коэффициентов и порога.
5. **Реализовать режим функционирования:**
   - Программа должна принимать на вход вектор из n чисел (0 или 1).
   - Сеть должна мгновенно выдавать результат (вероятность и округленный класс).
6. Сделать вывод о способности сигмоидального персептрона представлять базовые логические операции и восстанавливать полную таблицу истинности по частичной выборке.

Вариант 10

№ of variant	n	Task
10	7	AND

Код программы:

```
import numpy as np
import matplotlib.pyplot as plt
from itertools import product
```

```
n = 7


def logic_func(row):
    return int(all(v == 1 for v in row))


all_inputs = np.array(list(product([0, 1], repeat=n)))
all_outputs = np.array([logic_func(row) for row in all_inputs]).reshape(-1, 1)


print("\nТаблица истинности:")
for row, out in zip(all_inputs, all_outputs):
    print(f"{row.astype(int)} -> {int(out[0])}")

ones_idx = np.where(all_outputs[:, 0] == 1)[0]
zeros_idx = np.where(all_outputs[:, 0] == 0)[0]

np.random.seed(42)
zeros_perm = np.random.permutation(zeros_idx)
split = int(0.75 * len(zeros_idx))

train_idx = np.concatenate([ones_idx, zeros_perm[:split]])
test_idx = zeros_perm[split:]

X_train, y_train = all_inputs[train_idx], all_outputs[train_idx]
X_test, y_test = all_inputs[test_idx], all_outputs[test_idx]

print("\n" + "-" * 70)
print(f"Обучающая выборка: {len(train_idx)} наборов")
print(f"Тестовая выборка:  {len(test_idx)} наборов")
print("-" * 70)


def sigmoid(s):
    return 1.0 / (1.0 + np.exp(-np.clip(s, -500, 500)))


def linear_output(W, X, T):
    return X @ W - T


def bce_loss(y_pred, y_true):
    eps = 1e-12
    yp = np.clip(y_pred, eps, 1 - eps)
    return -np.sum(y_true * np.log(yp) + (1 - y_true) * np.log(1 - yp))


def fixed_fit(X_tr, y_tr, X_te, y_te, alpha=0.1, max_epochs=10000, Ee=0.01):
    N, n_local = X_tr.shape
    W = np.zeros((n_local, 1))
    T = 0.0
    Es_train, Es_test = [], []
    stopped = max_epochs

    print("\n[Фиксированный шаг обучения]")

    for ep in range(max_epochs):
        order = np.random.permutation(N)

        for idx in order:
            xi = X_tr[idx:idx + 1]
            ti = float(y_tr[idx, 0])
```

```
            s = float(linear_output(W, xi, T)[0, 0])
            yi = sigmoid(s)
            d = yi - ti

            W -= alpha * xi.T * d
            T += alpha * d

        p_tr = sigmoid(linear_output(W, X_tr, T))
        es_tr = bce_loss(p_tr, y_tr)
        Es_train.append(es_tr)

        p_te = sigmoid(linear_output(W, X_te, T))
        es_te = bce_loss(p_te, y_te)
        Es_test.append(es_te)

        if ep % 500 == 0:
            print(f"Эпоха {ep:5d} | train = {es_tr:.6f} | test = {es_te:.6f}")

        if es_tr <= Ee:
            stopped = ep + 1
            print(f"Обучение завершено на эпохе {stopped}")
            break

    return W, T, Es_train, Es_test, stopped


def adaptive_fit(X_tr, y_tr, X_te, y_te, max_epochs=10000, Ee=0.01):
    N, n_local = X_tr.shape
    W = np.zeros((n_local, 1))
    T = 0.0
    Es_train, Es_test = [], []
    stopped = max_epochs

    print("\n[Адаптивный шаг обучения]")

    for ep in range(max_epochs):
        order = np.random.permutation(N)

        for idx in order:
            xi = X_tr[idx:idx + 1]
            ti = float(y_tr[idx, 0])

            norm_sq = float(np.sum(xi ** 2))
            alpha_t = 1.0 / norm_sq if norm_sq > 1e-12 else 0.1

            s = float(linear_output(W, xi, T)[0, 0])
            yi = sigmoid(s)
            d = yi - ti

            W -= alpha_t * xi.T * d
            T += alpha_t * d

        p_tr = sigmoid(linear_output(W, X_tr, T))
        es_tr = bce_loss(p_tr, y_tr)
        Es_train.append(es_tr)

        p_te = sigmoid(linear_output(W, X_te, T))
        es_te = bce_loss(p_te, y_te)
        Es_test.append(es_te)

        if ep % 500 == 0:
            print(f"Эпоха {ep:5d} | train = {es_tr:.6f} | test = {es_te:.6f}")

        if es_tr <= Ee:
            stopped = ep + 1
```

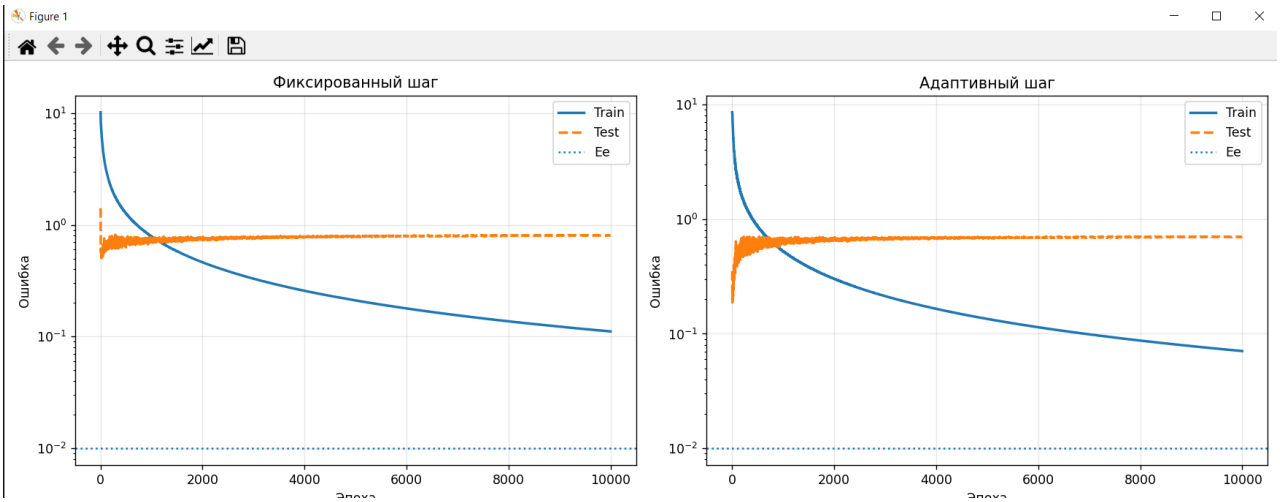
```
            print(f"Обучение завершено на эпохе {stopped}")
            break

    return W, T, Es_train, Es_test, stopped


Ee = 0.01
max_epochs = 10000

np.random.seed(42)
W_fix, T_fix, es_tr_fix, es_te_fix, ep_fix = fixed_fit(
    X_train, y_train, X_test, y_test, alpha=0.1, Ee=Ee, max_epochs=max_epochs
)

np.random.seed(42)
W_adp, T_adp, es_tr_adp, es_te_adp, ep_adp = adaptive_fit(
    X_train, y_train, X_test, y_test, Ee=Ee, max_epochs=max_epochs
)

print("\n" + "=" * 70)
print("ИТОГОВЫЕ ПАРАМЕТРЫ")
print("=" * 70)

print(f"\nФиксированный шаг:")
print(f"W = {W_fix.flatten()}")
print(f"T = {T_fix:.6f}")
print(f"Эпох = {ep_fix}")

print(f"\nАдаптивный шаг:")
print(f"W = {W_adp.flatten()}")
print(f"T = {T_adp:.6f}")
print(f"Эпох = {ep_adp}")

print("\nПроверка тестовой выборки (адаптивный режим):")
for xi, ti in zip(X_test, y_test):
    s = float(linear_output(W_adp, xi.reshape(1, -1), T_adp)[0, 0])
    prob = sigmoid(s)
    pred = 1 if prob >= 0.5 else 0
    print(f"{xi.astype(int)} -> {prob:.6f} -> класс {pred}")

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

for ax, es_tr, es_te, title in [
    (axes[0], es_tr_fix, es_te_fix, "Фиксированный шаг"),
    (axes[1], es_tr_adp, es_te_adp, "Адаптивный шаг"),
]:
    epochs = np.arange(1, len(es_tr) + 1)
    ax.plot(epochs, es_tr, linewidth=2, label="Train")
    ax.plot(epochs, es_te, linewidth=2, linestyle="--", label="Test")
    ax.axhline(Ee, linewidth=1.5, linestyle=":", label="Ee")
    ax.set_yscale("log")
    ax.set_xlabel("Эпоха")
    ax.set_ylabel("Ошибка")
    ax.set_title(title)
    ax.grid(True, alpha=0.3)
    ax.legend()

plt.tight_layout()
plt.show()

print(f"Введите {n} бинарных значений через пробел")
print("Для выхода: exit\n")

while True:
    user_input = input(">>> ").strip()
```

```
    if user_input.lower() in ("exit", ""):
        break

    try:
        vals = list(map(int, user_input.split()))
        if len(vals) != n or not all(v in (0, 1) for v in vals):
            raise ValueError

        point = np.array(vals, dtype=float).reshape(1, -1)
        s = float(linear_output(W_adp, point, T_adp)[0, 0])
        prob = sigmoid(s)
        cls = 1 if prob >= 0.5 else 0

        print(f"Сумма S = {s:+.6f}")
        print(f"Вероятность = {prob:.6f}")
        print(f"Класс = {cls}\n")

    except Exception:
        print(f"Ошибка: введите ровно {n} чисел 0/1\n")

print("Работа программы завершена.")
```

Результат программы:

```
Таблица истинности:
[0 0 0 0 0 0 0] -> 0
[0 0 0 0 0 0 1] -> 0
[0 0 0 0 0 1 0] -> 0
[0 0 0 0 0 1 1] -> 0
[0 0 0 0 1 0 0] -> 0
[0 0 0 0 1 0 1] -> 0
[0 0 0 0 1 1 0] -> 0
[0 0 0 0 1 1 1] -> 0
[0 0 0 1 0 0 0] -> 0
[0 0 0 1 0 0 1] -> 0
[0 0 0 1 0 1 0] -> 0
[0 0 0 1 0 1 1] -> 0
[0 0 0 1 1 0 0] -> 0
[0 0 0 1 1 0 1] -> 0
[0 0 0 1 1 1 0] -> 0
[0 0 0 1 1 1 1] -> 0
[0 0 1 0 0 0 0] -> 0
[0 0 1 0 0 0 1] -> 0
[0 0 1 0 0 1 0] -> 0
[0 0 1 0 0 1 1] -> 0
[0 0 1 0 1 0 0] -> 0
[0 0 1 0 1 0 1] -> 0
[0 0 1 0 1 1 0] -> 0
[0 0 1 0 1 1 1] -> 0
[0 0 1 1 0 0 0] -> 0
```

График:




```
[1 0 0 0 1 1 1] -> 0.000000 -> класс 0
[1 1 0 1 0 1 0] -> 0.000000 -> класс 0
[0 0 0 1 1 1 0] -> 0.000000 -> класс 0
[1 0 1 1 1 0 0] -> 0.000000 -> класс 0
[0 1 1 0 0 1 1] -> 0.000001 -> класс 0
[1 1 0 0 1 1 0] -> 0.000001 -> класс 0
Введите 7 бинарных значений через пробел
Для выхода: exit

>>> 0 1 1 1 0 0 0
Сумма S = -22.643377
Вероятность = 0.000000
Класс = 0
```

Сигмоидальный однослойный персептрон хорошо представляет базовые линейно разделимые логические операции, такие как AND и OR. В моем варианте сеть успешно обучилась функции AND от 7 переменных и смогла правильно определять значения не только на обучающей, но и на тестовой выборке. Это показывает, что по частичной выборке персептрон способен восстановить полную таблицу истинности за счет своей обобщающей способности. Для нелинейно разделимых функций возможностей однослойной сети уже недостаточно

Вывод: Проделав данную лабораторную работу я реализовал нейронную сеть (однослойный персептрон) для решения задач синтеза логических функций от n входов. Сравнил эффективность обучения при различных начальных условиях и параметрах шага.